

JavaScript Array Coding Practice: Full Solutions

1. Reverse an array in place without reverse()

```
function reverseInPlace(arr) {
  let left = 0, right = arr.length - 1;
  while (left < right) {
    [arr[left], arr[right]] = [arr[right], arr[left]];
    left++;
    right--;
  }
  return arr;
}
```

2. Rotate an array right by k steps

```
function rotateRight(arr, k) {
  const n = arr.length;
  k = k % n;
  return [...arr.slice(n - k), ...arr.slice(0, n - k)];
}
```

3. Remove all falsy values

```
function removeFalsy(arr) {
  return arr.filter(Boolean);
}
```

4. Merge two sorted arrays without sort()

```
function mergeSorted(a, b) {
  const result = [];
  let i = 0, j = 0;
  while (i < a.length && j < b.length) {
    if (a[i] < b[j]) result.push(a[i++]);
    else result.push(b[j++]);
  }
  return result.concat(a.slice(i), b.slice(j));
}
```

5. Find the second largest number

```
function secondLargest(arr) {
  let max = -Infinity, second = -Infinity;
  for (const n of arr) {
    if (n > max) { second = max; max = n; }
    else if (n > second && n < max) { second = n; }
  }
  return second;
}
```

6. Check if two arrays contain same elements (order doesn't matter)

```
function sameElements(a, b) {
  if (a.length !== b.length) return false;
  const count = {};
  a.forEach(x => count[x] = (count[x]||0) + 1);
  for (const x of b) {
    if (!count[x]) return false;
    count[x]--;
  }
  return Object.values(count).every(v => v === 0);
}
```

7. Two Sum: indices that add to target

```
function twoSum(nums, target) {
  const map = {};
  for (let i = 0; i < nums.length; i++) {
    const diff = target - nums[i];
    if (diff in map) return [map[diff], i];
    map[nums[i]] = i;
  }
}
```

8. Flatten nested array of arbitrary depth

```
function flatten(arr) {
  const res = [];
  for (const item of arr) {
    if (Array.isArray(item)) res.push(...flatten(item));
    else res.push(item);
  }
  return res;
}
```

9. Group strings into anagrams

```
function groupAnagrams(strs) {
  const map = {};
  for (const s of strs) {
    const key = s.split('').sort().join('');
    (map[key] = map[key]||[]).push(s);
  }
  return Object.values(map);
}
```

10. Longest increasing subsequence (n^2 DP)

```
function lis(arr) {
  const dp = Array(arr.length).fill(1);
  for (let i = 0; i < arr.length; i++) {
    for (let j = 0; j < i; j++) {
      if (arr[j] < arr[i]) {
        dp[i] = Math.max(dp[i], dp[j] + 1);
      }
    }
  }
  return Math.max(...dp);
}
```

11. Sort array of objects by date ascending

```
function sortByDate(arr) {  
  return arr.sort((a, b) => new Date(a.date) - new Date(b.date));  
}
```

12. Count occurrences of each element

```
function frequencyMap(arr) {  
  return arr.reduce((acc, x) => {  
    acc[x] = (acc[x]||0) + 1;  
    return acc;  
  }, {});  
}
```

13. Split array into chunks of size n

```
function chunk(arr, size) {  
  const res = [];  
  for (let i = 0; i < arr.length; i += size) {  
    res.push(arr.slice(i, i + size));  
  }  
  return res;  
}
```

14. Intersection of two arrays

```
function intersection(a, b) {  
  const setB = new Set(b);  
  return [...new Set(a.filter(x => setB.has(x)))];  
}
```

15. Find missing number from 1...n

```
function findMissing(nums) {  
  const n = nums.length + 1;  
  const expected = (n * (n + 1)) / 2;  
  const actual = nums.reduce((s, x) => s + x, 0);  
  return expected - actual;  
}
```

16. Implement map(), filter(), reduce()

```
Array.prototype.myMap = function(fn) {  
  const res = [];  
  for (let i = 0; i < this.length; i++) res.push(fn(this[i], i, this));  
  return res;  
};
```

```
Array.prototype.myFilter = function(fn) {  
  const res = [];  
  for (let i = 0; i < this.length; i++) {  
    if (fn(this[i], i, this)) res.push(this[i]);  
  }  
}
```

```

    return res;
};

Array.prototype.myReduce = function(fn, init) {
    let acc = init === undefined ? this[0] : init;
    for (let i = init === undefined ? 1 : 0; i < this.length; i++) {
        acc = fn(acc, this[i], i, this);
    }
    return acc;
};

```

17. Throttle processing of array elements

```

function throttleProcess(arr, callback, delay) {
    let i = 0;
    function next() {
        if (i < arr.length) {
            callback(arr[i++]);
            setTimeout(next, delay);
        }
    }
    next();
}

```

18. Max profit from one buy-sell

```

function maxProfit(prices) {
    let min = Infinity, profit = 0;
    for (const p of prices) {
        min = Math.min(min, p);
        profit = Math.max(profit, p - min);
    }
    return profit;
}

```

19. Shuffle array (Fisher–Yates)

```

function shuffle(arr) {
    for (let i = arr.length - 1; i > 0; i--) {
        const j = Math.floor(Math.random() * (i + 1));
        [arr[i], arr[j]] = [arr[j], arr[i]];
    }
    return arr;
}

```

20. All unique permutations of string

```

function permute(str) {
    const res = new Set();
    function helper(path, chars) {
        if (!chars.length) res.add(path);
        for (let i = 0; i < chars.length; i++) {
            helper(path + chars[i], chars.slice(0, i) + chars.slice(i + 1));
        }
    }
    helper('', str);
    return [...res];
}

```

21. Maximum subarray sum (Kadane's)

```
function maxSubarray(arr) {
  let maxSoFar = arr[0], maxEnd = arr[0];
  for (let i = 1; i < arr.length; i++) {
    maxEnd = Math.max(arr[i], maxEnd + arr[i]);
    maxSoFar = Math.max(maxSoFar, maxEnd);
  }
  return maxSoFar;
}
```

22. Remove duplicates without Set

```
function unique(arr) {
  const res = [];
  arr.forEach(x => {
    if (res.indexOf(x) === -1) res.push(x);
  });
  return res;
}
```

23. Merge overlapping intervals

```
function mergeIntervals(intervals) {
  if (!intervals.length) return [];
  intervals.sort((a, b) => a[0] - b[0]);
  const res = [intervals[0]];
  for (const [start, end] of intervals.slice(1)) {
    const last = res[res.length - 1];
    if (start <= last[1]) {
      last[1] = Math.max(last[1], end);
    } else {
      res.push([start, end]);
    }
  }
  return res;
}
```

24. Pivot index where left sum equals right sum

```
function pivotIndex(arr) {
  const total = arr.reduce((s, x) => s + x, 0);
  let left = 0;
  for (let i = 0; i < arr.length; i++) {
    if (left === total - left - arr[i]) return i;
    left += arr[i];
  }
  return -1;
}
```

25. Median of two sorted arrays in $O(\log n)$

```
function findMedianSorted(a, b) {
  if (a.length > b.length) return findMedianSorted(b, a);
  const m = a.length, n = b.length;
  let low = 0, high = m;
}
```

```

while (low <= high) {
  const i = (low + high) >> 1;
  const j = Math.floor((m + n + 1) / 2) - i;
  const maxL = i === 0 ? -Infinity : a[i-1];
  const minR = i === m ? Infinity : a[i];
  const maxL2 = j === 0 ? -Infinity : b[j-1];
  const minR2 = j === n ? Infinity : b[j];
  if (maxL <= minR2 && maxL2 <= minR) {
    if ((m + n) % 2) return Math.max(maxL, maxL2);
    return (Math.max(maxL, maxL2) + Math.min(minR, minR2)) / 2;
  } else if (maxL > minR2) high = i - 1;
  else low = i + 1;
}
}

```

26. Trapping rainwater

```

function trap(height) {
  let left = 0, right = height.length - 1;
  let maxL = 0, maxR = 0, water = 0;
  while (left < right) {
    if (height[left] < height[right]) {
      maxL = Math.max(maxL, height[left]);
      water += maxL - height[left];
      left++;
    } else {
      maxR = Math.max(maxR, height[right]);
      water += maxR - height[right];
      right--;
    }
  }
  return water;
}

```

27. Schedule tasks to maximize completed tasks

```

function maxTasks(tasks) {
  tasks.sort((a, b) => a.deadline - b.deadline);
  let time = 0;
  const maxHeap = [];
  for (const {duration, deadline} of tasks) {
    time += duration;
    maxHeap.push(duration);
    maxHeap.sort((a, b) => b - a);
    if (time > deadline) {
      time -= maxHeap.shift();
    }
  }
  return maxHeap.length;
}

```

28. Convert sparse to full array

```

function sparseToFull(tuples, length) {
  const arr = Array(length).fill(null);
  for (const [i, v] of tuples) {
    arr[i] = v;
  }
  return arr;
}

```

29. Implement queue with two stacks

```
class MyQueue {
  constructor() {
    this.in = [];
    this.out = [];
  }
  enqueue(x) {
    this.in.push(x);
  }
  dequeue() {
    if (!this.out.length) {
      while (this.in.length) this.out.push(this.in.pop());
    }
    return this.out.pop();
  }
}
```

30. Top k frequent words

```
function topKFrequent(words, k) {
  const freq = {};
  words.forEach(w => freq[w] = (freq[w] || 0) + 1);
  return Object.keys(freq)
    .sort((a, b) => freq[b] - freq[a] || a.localeCompare(b))
    .slice(0, k);
}
```